

PROJETO ARDUINO – Parte 01

Prof. Sergio Luiz da Silva



PROJETO ARDUINO – Parte 01

LENDO UM BOTÃO



LENDO UM BOTÃO

Agora que você já é capaz de acionar cargas simples através do seu Arduino, vamos aprender a trabalhar com o nosso primeiro sensor, o **BOTÃO**.

O botão é um dos dispositivos mais populares quando se fala de interface humano-máquina.

LENDO UM BOTÃO

Seja no seu smartphone, nos elevadores ou até mesmo o próprio interruptor da luz da sua sala, os botões são responsáveis por disparar ações em diversos sistemas eletrônicos presentes no nosso dia-a-dia.

LENDO UM BOTÃO

Com um simples apertar você pode ligar uma lâmpada, travar a tela do seu celular ou interagir com um jogo eletrônico.

Vamos então entender mais sobre esse recurso simples e barato, que é extremamente versátil e será utilizado em diversos exemplos ao longo dos projetos.

LENDO UM BOTÃO

ENTRADAS DIGITAIS

No projeto anterior você viu como os **pinos do Arduino** funcionam como saída digital, onde a energia flui por eles até a carga.

Entretanto, eles também podem ser configurados para fazer o contrário, deixando com que a energia flua de fora para dentro, podendo então ser mensurada.

LENDO UM BOTÃO

ENTRADAS DIGITAIS

Assim fica fácil detectar quando a tensão no pino é **5V** e quando é **0V**.

Então quando queremos usar um botão com nossa placa, basta conectá-lo corretamente a um pino que esteja configurado como entrada e poderemos detectar se o botão está pressionado ou solto.

LENDO UM BOTÃO

RESISTOR DE *PULL-DOWN*

Se você **simplesmente conectar um dos pares de terminais ao VCC e o outro ao pino do Arduino**, quando você pressionar o botão o pino terá **5V**.

Mas qual sinal chegará ao pino do Arduino quando o botão não estiver pressionado? Nada?

LENDO UM BOTÃO

RESISTOR DE *PULL-DOWN*

Na verdade a melhor resposta seria "**qualquer coisa**", pois como não tem nenhum sinal válido (**5V** ou **0V**) conectado a ele, o caminho fica livre para os ruídos existentes no circuito.

O **pino do Arduino** então poderá assumir um valor lógico **VERDADEIRO** ou **FALSO** a qualquer momento.

LENDO UM BOTÃO

RESISTOR DE *PULL-DOWN*

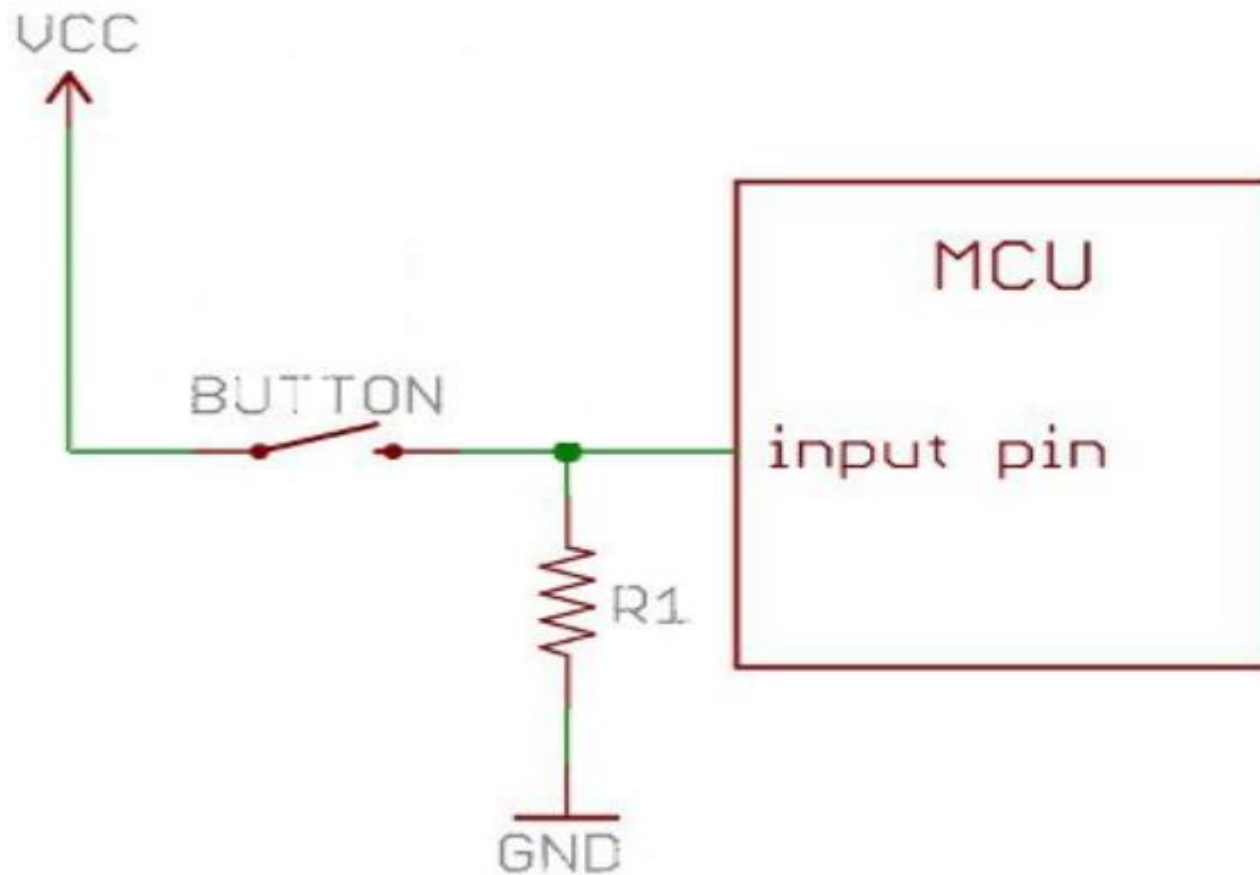
Para retirar essas flutuações que prejudicam a interpretação dos sinais, nós utilizamos normalmente um **RESISTOR DE PULL-DOWN**.

Ele atua garantindo que o sinal de GND chegue até o pino enquanto o botão não for pressionado, e protege o circuito contra um curto circuito quando você aperta o botão.

LENDO UM BOTÃO

RESISTOR DE *PULL-DOWN*

Veja um exemplo de ligação na imagem abaixo.



LENDO UM BOTÃO

DESENVOLVENDO O PROJETO

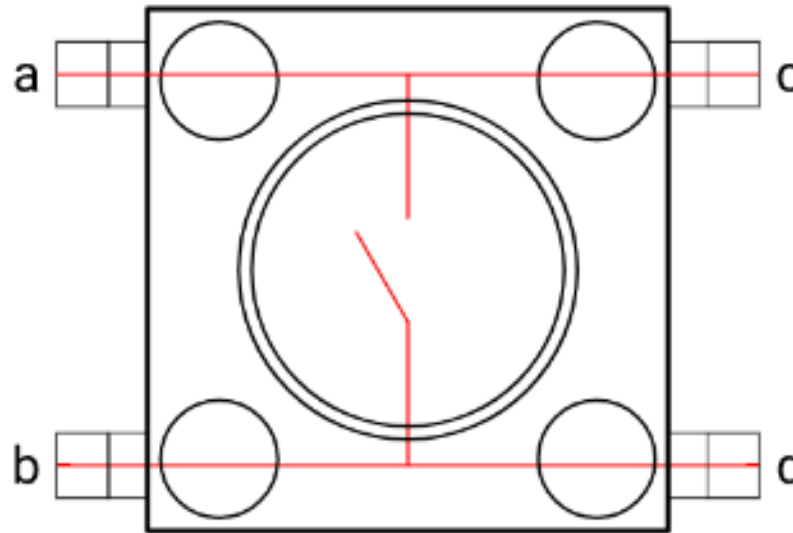
Vamos usar um botão para controlar o estado de um LED. O botão que possui quatro terminais são interligados aos pares (terminal **a** com **c** e **b** com **d**).

LENDO UM BOTÃO

DESENVOLVENDO O PROJETO

Ao pressionar o botão, os quatro terminais são conectados entre si, gerando continuidade no circuito.

Podemos então utilizar esse funcionamento para alterar o estado lógico de uma entrada digital do Arduino.



PRATICANDO

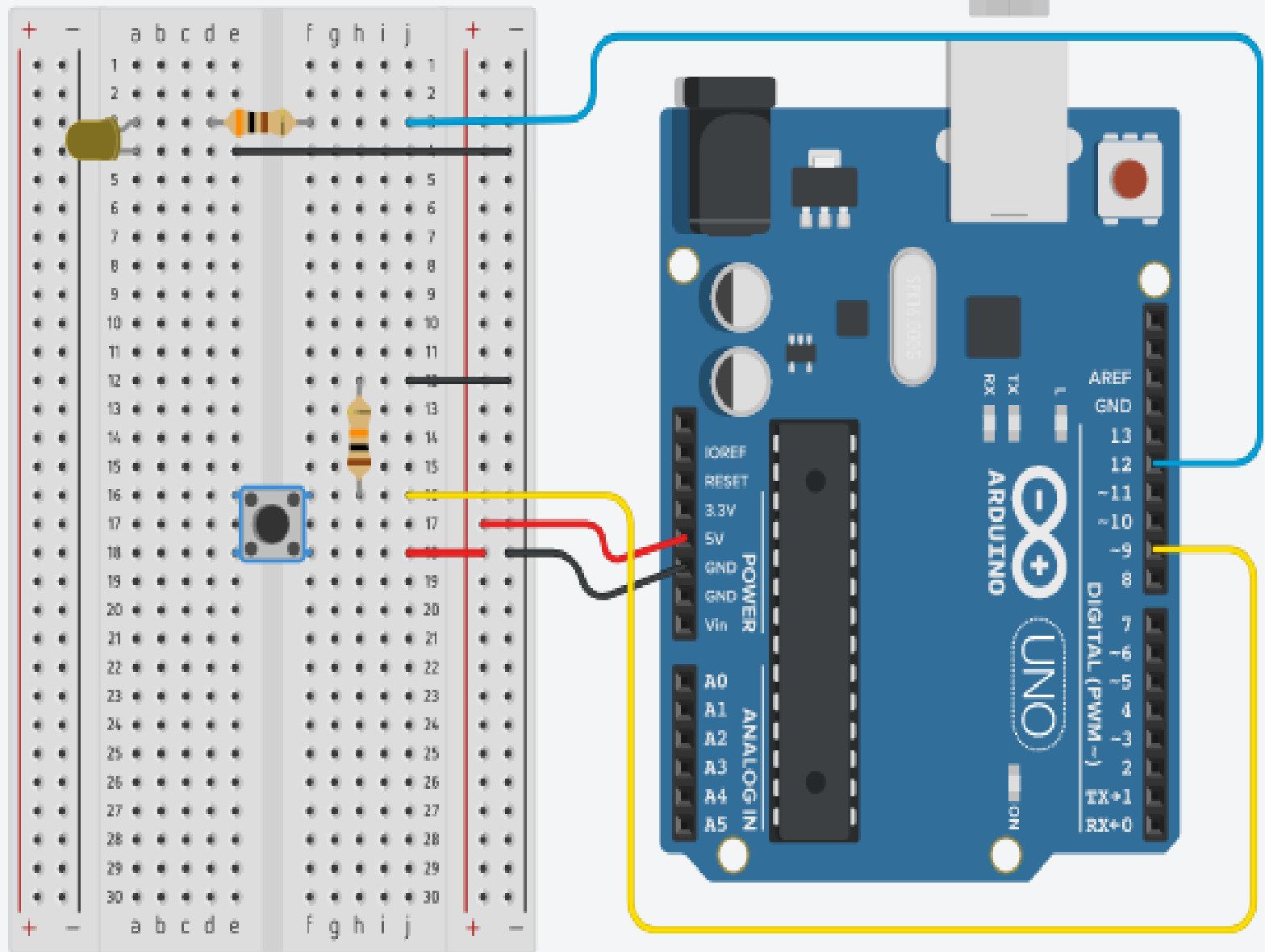
LISTA DE MATERIAIS

Você precisará dos seguintes materiais para o desenvolvimento dessa atividade:

- 1x **Placa Arduino + Cabo USB**
- 1x **LED 5mm**
- 1x **Resistor 300Ω**
- 1x **Chave Momentânea (PushButton)**
- 1x **Resistor 10KΩ**
- 1x **Protoboard**
- **Jumpers**

LENDO UM BOTÃO

Colocando um resistor para limitar a corrente do **LED** e um resistor de pull-down no **BOTÃO**, podemos então montar o circuito para esse exemplo conforme a imagem ao lado:



MONTANDO no ARDUINO

LENDO UM BOTÃO

Desconecte a placa de seu computador e monte o circuito conforme montou no TINKERCAD. Lembre-se do que falamos sobre os LEDs e resistores para conectá-los corretamente em sua protobard.



LEND O UM BOTÃO

Desenvolvendo o **CÓDIGO** para testar.

Ao carregar o código do próximo slide em seu Arduino, você poderá ver o LED acender quando pressionar o botão e apagar quando soltar.

LEND O UM BOTÃO

Desenvolvendo o **CÓDIGO** para testar.

```
1. /*****
2. * Projeto Botão Push
3. * Acende o LED quando o botão é pressionado
4. * e o apaga quando o botão é solto.
5. *****/
6.
7. void setup() {
8.   pinMode(9, INPUT); // configura o pino com o
   botão como entrada
9.   pinMode(12, OUTPUT); // configura o pino com
   o LED como saída
10. }
```

CONTINUA...

LENDO UM BOTÃO

...CONTINUAÇÃO

```
11.  
12. void loop() {  
13.   if (digitalRead(9) == HIGH) { // se botão  
    estiver pressionado (HIGH)  
14.     digitalWrite(12, HIGH); // acende o LED  
15.   }  
16.   else { // se não estiver pressionado (LOW)  
17.     digitalWrite(12, LOW); // apaga o LED  
18.   }  
19. }
```

FIM

LENDO UM BOTÃO

ENTENDENDO O CÓDIGO

Você já tinha aprendido anteriormente a configuração do pino do **LED** como saída e o seu acionamento.

Para configurar o pino do botão como entrada, também utilizamos a função **pinMode**, só que dessa vez usamos **INPUT** no lugar de **OUTPUT**.

LENDO UM BOTÃO

ENTENDENDO O CÓDIGO

E para realizar a "**escrita digital**" no pino do **LED** usamos **digitalWrite**, enquanto que para realizar a leitura digital se usa **digitalRead**.

Ao contrário da **função de escrita, que não retorna nenhum valor**, a **função de leitura nos retorna o valor atual do pino entre parênteses (nesse caso, o 9)**.

LEND O UM BOTÃO

ENTENDENDO O CÓDIGO

Isso significa que **se o botão estiver pressionado, o valor da tensão no pino é de 5V** e a função retornará **HIGH**.

Enquanto o **botão estiver solto, o valor é de 0V**, retornando então **LOW**.

LENDO UM BOTÃO

CRIANDO CONDIÇÕES

Até o momento só tínhamos construído códigos sequenciais, que sempre executavam uma linha atrás da outra.

Agora nós precisamos realizar algum tipo de desvio, pois acender ou apagar o **LED** depende de uma **condição**: o botão estar ou não pressionado.

LENDO UM BOTÃO

CRIANDO CONDIÇÕES

É para isso que entra a estrutura condicional **if...else**.

Ela **realiza um teste lógico** para **decidir entre dois trechos de código**.

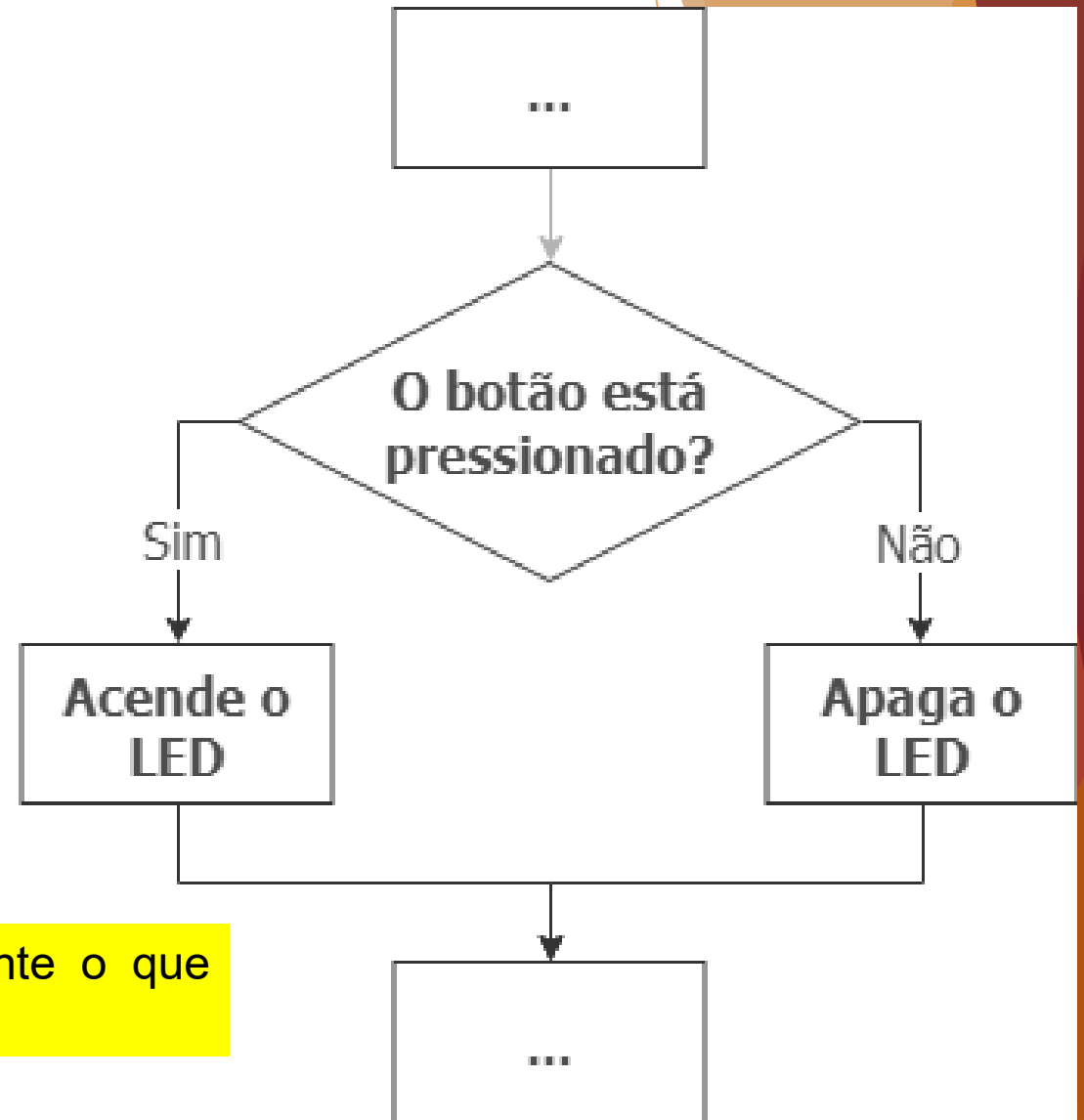
Ela executa o primeiro se a condição for verdadeira e o segundo se a condição for falsa.

LENDO UM BOTÃO

CRIANDO CONDIÇÕES

```
1 if (condição){  
2   // código executado se a condição for verdadeira (HIGH)  
3 }  
4 else{  
5   // código executado se a condição for falsa (LOW)  
6 }
```

Ao lado temos um fluxograma que explica graficamente o que acontece durante a execução do nosso código.



LENDO UM BOTÃO

COMPARANDO COISAS

Quando começamos a realizar testes lógicos, é natural que precisemos comparar as coisas.

Seja dois valores, resultados de funções, variáveis ou quaisquer outras grandezas, podemos utilizar os **operadores de comparação** para definir qual é a relação entre eles.

LENDO UM BOTÃO

COMPARANDO COISAS

Veja na tabela abaixo **quais são os operadores de comparação** aceitos pela linguagem do Arduino e alguns exemplos.

Símbolo	Comparação	Exemplo
!=	não igual a	HIGH != LOW
<	menor que	1 < 2
<=	menor ou igual a	2 <= 4
==	Igual a	HIGH == HIGH
>	maior que	16 > 8
>=	maior ou igual a	32 >= 32

LENDO UM BOTÃO

COMPARANDO COISAS

Em nosso código utilizamos o operador **==** para verificar se o resultado da **função digitalRead(9)** é igual a **HIGH**.

Somente quando essa comparação é verdadeira é que o **LED acende**.

FIM!